SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai- 602105

Student Name: Rachitha R **Reg. No.:** 192421058

Course Code: CSA1616 **Slot:** D

Course Name: Data Warehouse and data Mining

Course Faculty: Dr. Mahesh Muthulakshmi

EXP:01 To find intervals and corresponding Frequencies.

Aim:

To find the intervals and corresponding frequencies from the given age and frequency data.

Procedure:

1. Open R/RStudio and create a new R script.
2. Define the given age values using the c() function and store them in the variable age.
3. Define the corresponding frequency values using the c() function and store them in the variable frequency.
4. Use the median() function to calculate the median of the given age values.
5. Use the median() function to calculate the median of the given frequency values.
6. Execute the program and observe the results in the R Console.

Program:

```
1  age <- c(5,15,20,25,59,76,87)
2  frequency <- c(43, 45, 76, 87, 98, 99)
3  median(age)
4  median(frequency)
```

Output:

```
> age <- c(5,15,20,25,59,76,87)
> frequency <- c(43, 45, 76, 87, 98, 99)
> median(age)
[1] 25
> median(frequency)
[1] 81.5
```

Result:

The given age and frequency data were successfully analyzed using R. The median of the age values is 25, and the median of the frequency values was calculated successfully.

192421058

EXP:02 To find the data for analysis includes the attribute age.

Aim:
To find the mean, median, mode, range and quartiles for the given age data.

Procedure:

1. Open **R/RStudio** and create a new R script.
2. Define the given age values using the c() function and store them in the variable age.
3. Use the mean() function to calculate the mean of the age values.
4. Use the median() function to calculate the median of the age values.
5. Use the table() function to find the frequency of each age value and determine the mode.
6. Use the range() function to find the minimum and maximum values of the age data.
7. Use the quantile() function with 0.25 to find the first quartile (Q1).
8. Use the quantile() function with 0.75 to find the third quartile (Q3).
9. Execute the program and observe the results in the R Console.

Program:

```
1
2  age <- c(13,15,16,16,19,20,20,21,22,22,25,25,25,25,
3           30,33,33,35,35,35,35,36,40,45,46,52,70)
4
5  mean(age)
6
7  median(age)
8
9  mode_age <- names(table(age))[table(age) == max(table(age))]
0  mode_age
1
2  range(age)
3
4  quantile(age, 0.25)
5
6  quantile(age, 0.75)
```

Output:

```
> age <- c(13,15,16,16,19,20,20,21,22,22,25,25,25,25,
+          30,33,33,35,35,35,35,36,40,45,46,52,70)
> mean(age)
[1] 29.96296
> median(age)
[1] 25
> mode_age <- names(table(age))[table(age) == max(table(age))]
> mode_age
[1] "25" "35"
> range(age)
[1] 13 70
> quantile(age, 0.25)
 25%
20.5
> quantile(age, 0.75)
75%
 35
```

Result:
The given age data was successfully analyzed using R. The mean, median, mode, range, first

quartile (Q1), and third quartile (Q3) were calculated successfully.

EXP:03 To Show Data Preprocessing in Reduction and Transformation.

Aim:

To show data preprocessing using reduction and transformation techniques.

Procedure:

1. Open **R/RStudio** and create a new R script.
2. Enter the given data values using the c() function and store them in the variable data.
3. Specify the number of bins as **5** for grouping the data.
4. Use the cut() function to divide the data into five intervals or bins.
5. Use the tapply() function with mean to perform **mean smoothing** for each bin.
6. Use the tapply() function with median to perform **median smoothing** for each bin.
7. Use the tapply() function with min() and max() to perform **min-max smoothing** for each bin.
8. Display the calculated smoothed values using the print() function.
9. Execute the program and observe the results in the R Console

Program:

```
data <- c(11,13,13,15,15,16,19,20,20,20,21,21,
          22,23,24,30,40,45,45,45,71,72,73,75)
bins <- 5
bin_indices <- cut(data, bins)
mean_smooth <- tapply(data, bin_indices, mean)
print(mean_smooth)
median_smooth <- tapply(data, bin_indices, median)
print(median_smooth)
min_max_smooth <- tapply(data, bin_indices, function(x) c(min(x), max(x)))
print(min_max_smooth)
```

Output:

```
> data <- c(11,13,13,15,15,16,19,20,20,20,21,21,
+          22,23,24,30,40,45,45,45,71,72,73,75)
> bins <- 5
> bin_indices <- cut(data, bins)
> mean_smooth <- tapply(data, bin_indices, mean)
> print(mean_smooth)
(10.9,23.8] (23.8,36.6] (36.6,49.4] (49.4,62.2] (62.2,75.1]
   17.78571    27.00000    43.75000          NA    72.75000
> median_smooth <- tapply(data, bin_indices, median)
> print(median_smooth)
(10.9,23.8] (23.8,36.6] (36.6,49.4] (49.4,62.2] (62.2,75.1]
       19.5        27.0        45.0          NA        72.5
> min_max_smooth <- tapply(data, bin_indices, function(x) c(min(x), max(x)))
> print(min_max_smooth)
$`(10.9,23.8]`
[1] 11 23

$`(23.8,36.6]`
[1] 24 30

$`(36.6,49.4]`
[1] 40 45

$`(49.4,62.2]`
NULL

$`(62.2,75.1]`
[1] 71 75
```

Result:

The given data was successfully preprocessed using mean smoothing, median smoothing, and min-max smoothing techniques by dividing the data into five bins.